Hiding in JPEG DCT Coefficient Parity

Group 16 – Dominic Miller, Carlos Mejia Rosales, Fiza Rahman

CS 4463 – Steganography, August 2026

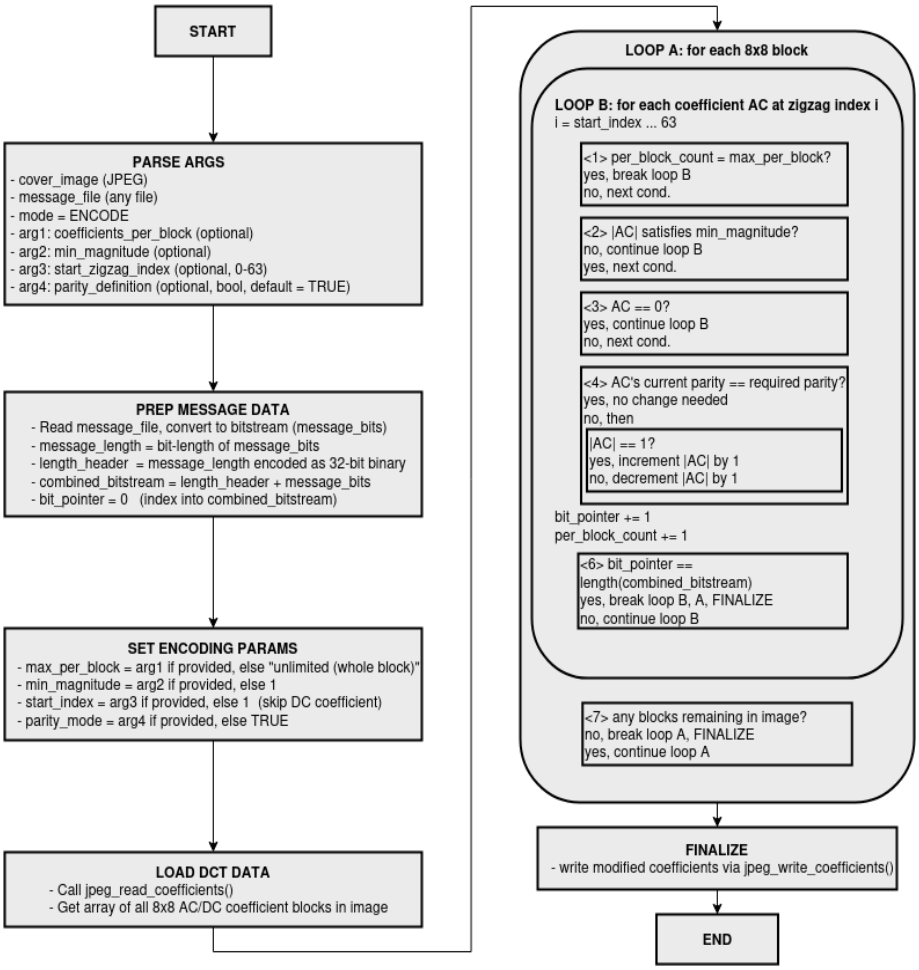
Introduction

We chose Option #1 for our project: Creating a Steganographic encoding and decoding tool. The goal of our tool is to be able to hide and extract bits of a hidden message in the parity of JPEG DCT coefficients. Our general approach is as follows: The user will execute the encode command for the program with the cover file, and the message file. The program converts the message text to a bitstream to prepare it for embedding, and calculate its length. The message will be hidden after the quantization process so the data can be preserved. Each 8x8 block of the JPEG will be examined sequentially to see if its AC coefficients fit the standards (which may be changed by the user) to be modified for data hiding. If the magnitude of the AC coefficient is greater than 0, it meets the default qualifications. AC coefficients equal to 0 are avoided by default because incrementing them will drastically hinder the effectiveness of the final Run-Length Encoding and Huffman / Arithmetic Coding, and increase file size notably. Before any message bits are hidden, a 32-bit number will be embedded in the first 32 available AC coefficients (a technique used in f5 as well). This will store the size of the message, so the decoder will know when to stop reading. Bits are hidden in AC coefficient candidates by decrementing (or incrementing, when the coefficient is 1) the value of the DCT coefficient so that the parity of consecutive coefficients will align with the values of binary strings. For example, say there are 5 candidate AC coefficients in an 8x8 block: [2, 3, 2, 5, 4]. The message we will hide here is 10011. For this example, even will be represented by 0. So, 2 will be decremented to 1, 3 → 2, 2 remains the same, 5 remains the same, 4 → 3. This hiding process will be repeated until the hiding capacity of the JPEG is expended, or the message is complete. Arguments will be available to alter: the number of acceptable affected coefficients per block, the minimum magnitude of affected coefficients, which index of the DCT coefficient matrix to start hiding data on (assuming zigzag traversal), and parity definition (what parity bits correspond to). The decoding process is similar, and will require all the same arguments passed for the encoding process to be properly decoded. The DCT coefficients for each block are read according to the hiding scheme used in the encoding process, and each bit is written to the file consecutively. This continues until there are no more blocks left, or the message length recorded in the 32-bit header is reached, whichever comes first. After obtaining stegoimages using this method, the results of our analysis showed that this program produces relatively high capacity images, with minimal distortion. Perceptible changes scale inversely with the size of the image, as there can only be so many bits written per pixel, meaning smaller images will have greater distortion than larger images.

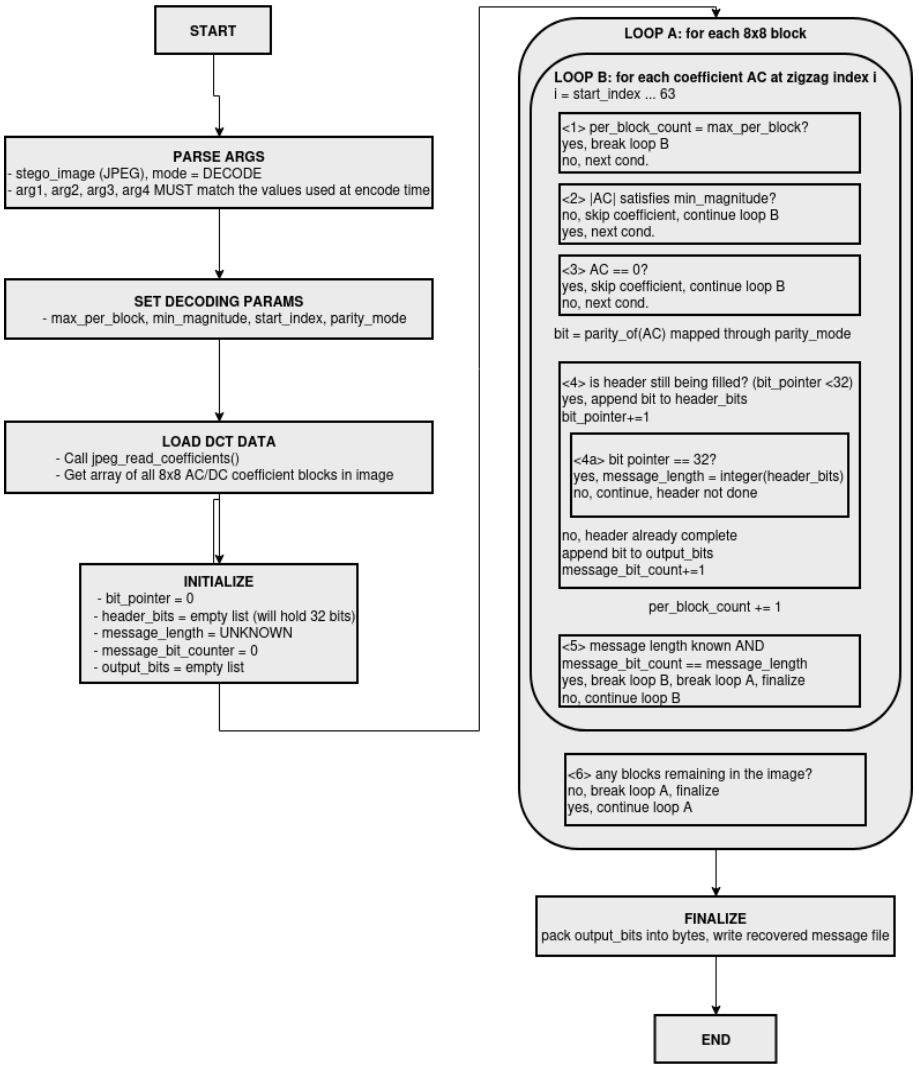
Discussion

The full program is conceptually simple, robust, and leverages JPEG's DCT coefficient tables to produce stegoimages with relatively high capacity and little perceptual change. The embedding (stego_embed.c) and extracting (stego_extract.c) algorithms use a number of functions from libjpeg. Rather than adopt, and attempt to fully grasp libjpeg's features, the embedding and extraction algorithms need only use a narrow subset of libjpegs full toolset, which was adapted using jpeg_wrapper.c. This portion of the project defines several libjpeg functions into functions that are convenient for the use case of this program. The embedding algorithm reads the message file into a heap-allocated buffer, as well as the message file's size in bytes. The latter is embedded into the first 32 bits of the cover image, so that when the extraction algorithm reads the message, it knows where to stop. The cover image's DCT tables are parsed, and the message bits are embedded into every valid coefficient (depending on what the user defines as "valid") by checking if the parity of the coefficient matches the corresponding message bit, and changing the parity if it doesn't. After the process is complete, the changes are all written to the cover image's DCT tables at the same time. The extraction algorithm works in a similar way, but in reverse. The DCT tables of the stegoimage are traversed through, and bits matching the scheme defined in the embedding process are written to recover the message file. Crucially, the same arguments with the same values must be passed to both the embedding and extraction algorithms for the message to be read correctly. The area we faced the most technical difficulties in was the jpeg_wrapper code. Understanding which functions to use in the libjpeg library, and implementing them in a way that correctly interfaced with the library and fit our goals was difficult. This was also the area where C memory management was most difficult. However, libjpeg is very well document, and the code is commented thoroughly, so the task was surmountable. In the end, we were able to create an effective libjpeg wrapper for our project that made it far more simple to use the features of libjpeg that we wanted to within the scope of our code. Features that go beyond the basic approach would be the -m, -s, and -p flags: minimum DCT magnitude, zigzag starting index, and parity mapping respectively. The minimum DCT magnitude can be set to 0, greatly bloating the file size by interfering with the (usually) many zeroes in the DCT tables, and worsening the effectiveness of the Huffman encoding. The zigzag starting index can also be set to 0, which allows the DC coefficient of each table to be affected. This is usually not recommended, but is interesting to play with. The parity mapping flag isn't particularly useful, but might have a niche use-case, and was easy to implement. For future work, a way to change the parity of each digit in a base-10 DCT coefficient could greatly increase the capacity, and add new ways to increase the visibility of the approach.

Endcoding Process
Fucntional Block Diagram



Decoding Process
Functional Block Diagram



Statistical Analysis

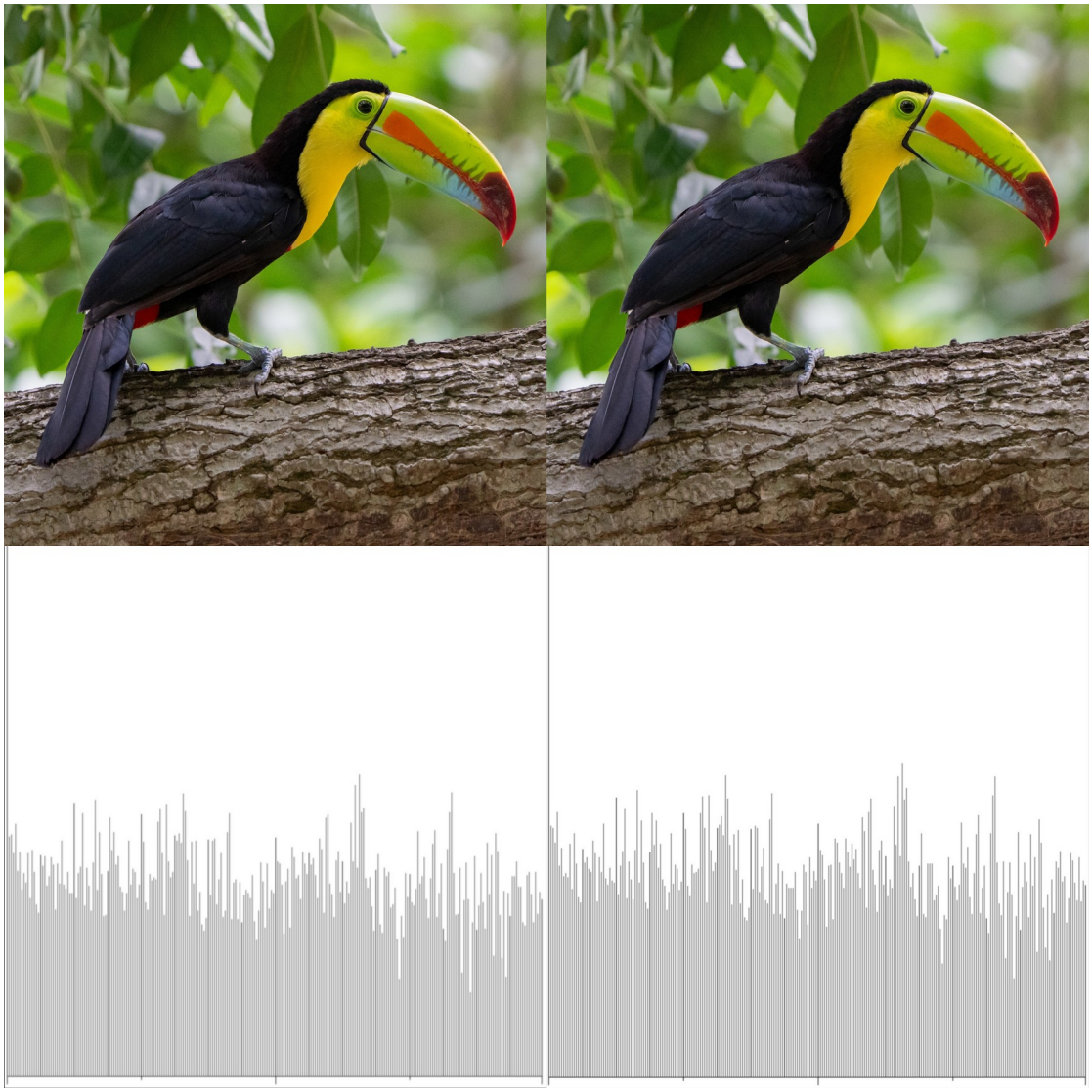
To evaluate the detectability of our DCT parity steganography algorithm, we performed a basic statistical analysis using the Write Bitmap Histogram (wbh_v6.35). We compared the basic cover images with several stego images using different payloads. The goal was to determine whether the embedded data produced noticeable changes in the byte histogram.

Test Images

Images	**Embedded Data**
toucan.jpg	Original
toucan_basic_stego.jpg	Small text message
toucan_1000.jpg	1000 random bytes
toucan_5000.jpg	5000 random bytes
toucan_options.jpg	Small message w/ custom parameters
zorn.jpg	Original
zorn_basic_stego.jpg	Small text message
zorn_random1000.jpg	1000 random bytes
zorn_random5000.jpg	5000 random bytes
zorn_test_options.jpg	Custom parameters

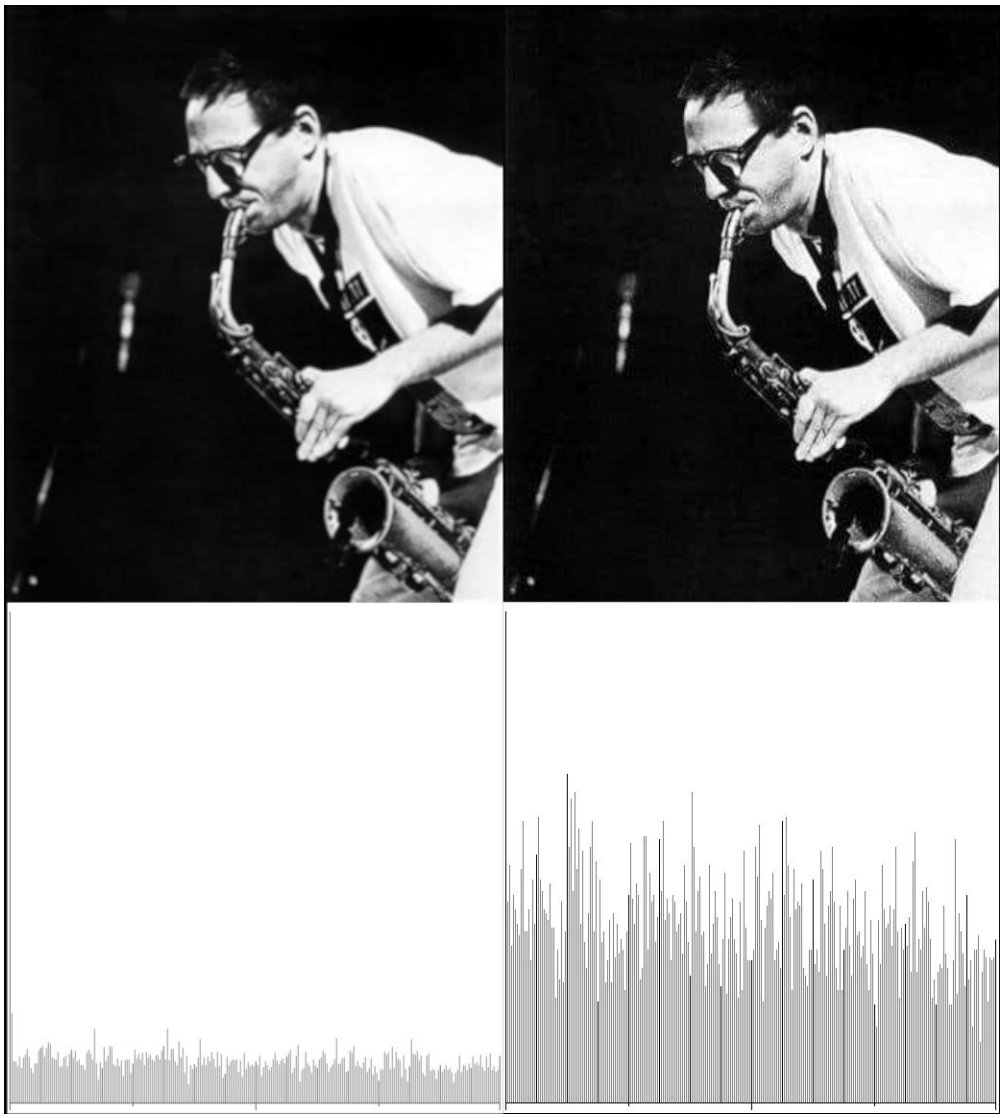
Results

toucan.jpg | toucan_5000.jpg



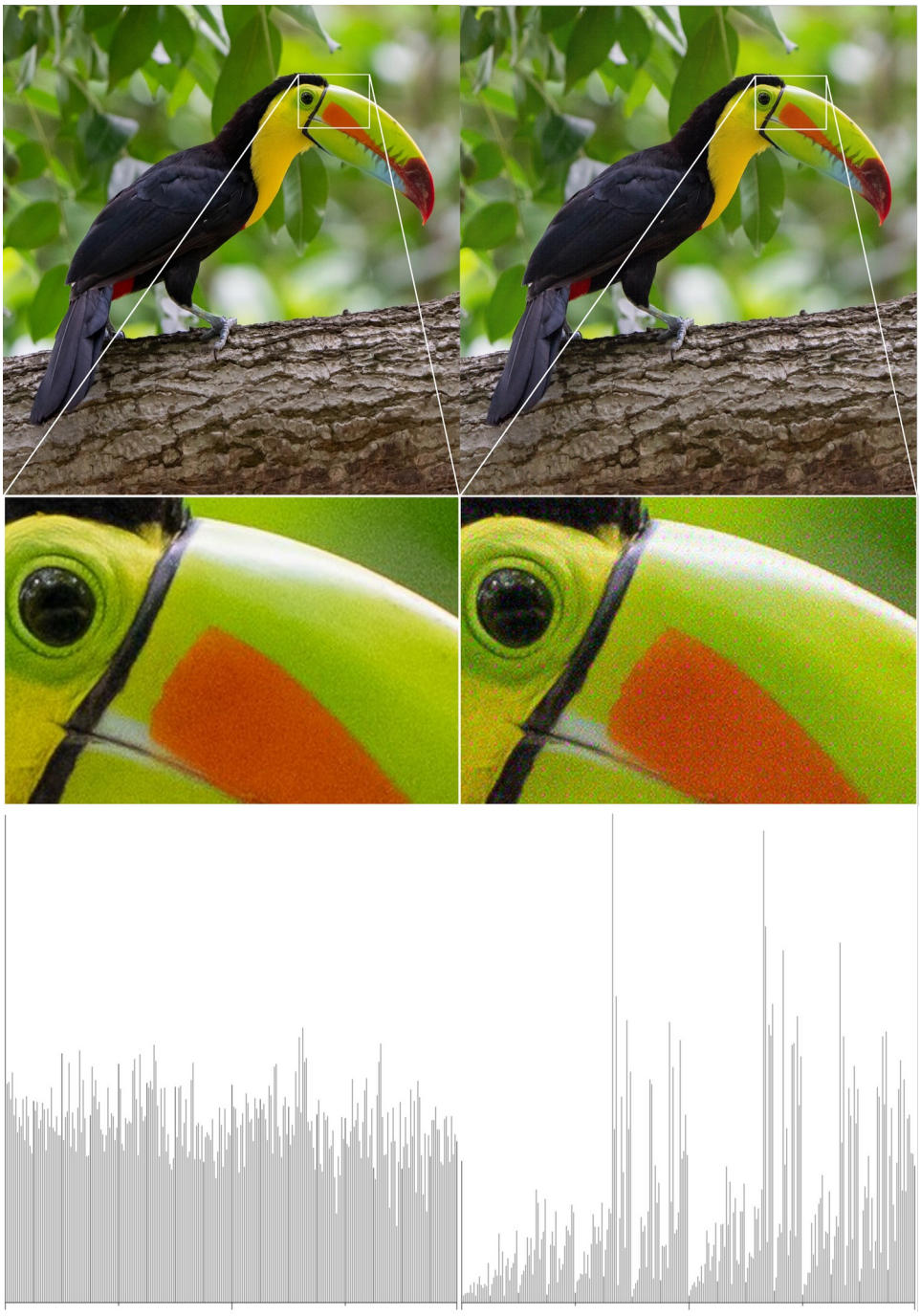
The histogram of the toucan image changed very little after embedding, even with a 5000-byte payload, the distribution remained visually similar to the original, suggesting that the algorithm introduces minimal statistical artifacts. The overall histogram shape remained similar and did not clearly reveal hidden data.

zorn.jpg zorn_random5000.jpg



The smaller Zorn image showed slightly larger changes than the toucan image because the payload represents a larger percentage of the file. However, the overall histogram shape remained similar and did not clearly reveal hidden data. The visual change in the histograms is due to the large spike of zeroes in the original image being overwritten with random bytes, changing the scale of the image.

toucan.jpg toucan-marmoset.jpg



This image shows the distortion and histograms of the toucan image after embedding it with the marmoset image in samples/, with minimum absolute DCT coefficient magnitude being 0, and the zigzag starting index being 0 (-m 0 -s 0) for the greatest visual perception. The distortion is apparent upon close inspection, but not obvious.

Numerical Results

Toucan

Image	Entropy	RatioH	Std Dev
Original	7.9709	0.9964	332.16%
Basic	7.9728	0.9966	320.86%
Random 1000	7.9712	0.9964	320.14%
Random 5000	7.9727	0.9966	320.95%
Options	7.9708	0.9964	332.91%

Zorn

Image	Entropy	RatioH	Std Dev
Original	7.8735	0.9842	1056.27%
Basic	7.9523	0.9940	432.75%
Random 1000	7.9411	0.9926	475.49%
Random 5000	7.9420	0.9927	463.69%
Options	7.9430	0.9929	487.04%

Tools Used

wbh_v6.35.exe

Bibliography:

Paruchuri, Jithendra K., and Sen-Ching S. Cheung. "Joint optimization of data hiding and video compression." ISCAS. 2008.

Fridrich, Jessica, Miroslav Goljan, and Dorin Hogea. "Steganalysis of JPEG images: Breaking the F5
algorithm." International Workshop on Information Hiding. Berlin, Heidelberg: Springer Berlin Heidelberg,2002.

Kodovský, Jan, and Jessica Fridrich. "Quantitative structural steganalysis of Jsteg." IEEE Transactions on Information Forensics and Security 5.4 (2010): 681-693.